

TransformerDecoderLayer#

[https://pytorch.org/docs/stable/generated/torch.nn.TransformerDecoderLayer.h](https://pytorch.org/docs/stable/generated/torch.nn.TransformerDecoderLayer.html)

Description:

TransformerDecoderLayer is made up of self-attn, multi-head-attn and feedforward network. This TransformerDecoderLayer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem. `d_model` (int) – the number of expected features in the input (required). `nhead` (int) – the number of heads in the multiheadattention models (required). `dim_feedforward` (int) – the dimension of the feedforward network model (default=2048).

Function Signature

```
class torch.nn.TransformerDecoderLayer(d_model, nhead,  
dim_feedforward=2048, dropout=0.1, activation=<function  
relu>, layer_norm_eps=1e-05, batch_first=False,  
norm_first=False, bias=True, device=None, dtype=None)  
[source]#
```

Parameters

Alternatively, when `batch_first` is `True`:

```
forward(tgt, memory, tgt_mask=None, memory_mask=None,  
tgt_key_padding_mask=None, memory_key_padding_mask=None,  
tgt_is_causal=False, memory_is_causal=False)[source]#
```

Parameters

Returns:

Tensor

Code Examples

```
>>> decoder_layer = nn.TransformerDecoderLayer(d_model=512,
nhead=8)
>>> memory = torch.rand(10, 32, 512)
>>> tgt = torch.rand(20, 32, 512)
>>> out = decoder_layer(tgt, memory)
```

```
>>> decoder_layer = nn.TransformerDecoderLayer(
...     d_model=512, nhead=8, batch_first=True
... )
>>> memory = torch.rand(32, 10, 512)
>>> tgt = torch.rand(32, 20, 512)
>>> out = decoder_layer(tgt, memory)
```

Detailed Documentation

Documentation

`TransformerDecoderLayer` is made up of self-attn, multi-head-attn and feedforward network.

This `TransformerDecoderLayer` implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient

layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

`d_model` (int) – the number of expected features in the input (required).

`nhead` (int) – the number of heads in the multiheadattention models (required).

`dim_feedforward` (int) – the dimension of the feedforward network model (default=2048).

`dropout` (float) – the dropout value (default=0.1).

`activation` (Union[str, Callable[[Tensor], Tensor]]) – the activation function of the intermediate layer, can be a string (“relu” or “gelu”) or a unary callable. Default: relu

`layer_norm_eps` (float) – the eps value in layer normalization components (default=1e-5).

`batch_first` (bool) – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False (seq, batch, feature).

`norm_first` (bool) – if True, layer norm is done prior to self attention, multihead attention and feedforward operations, respectively. Otherwise it’s done after. Default: False (after).

`bias` (bool) – If set to False, Linear and LayerNorm layers will not learn an additive bias. Default: True.

Pass the inputs (and mask) through the decoder layer.

`tgt` (Tensor) – the sequence to the decoder layer (required).

`memory` (Tensor) – the sequence from the last layer of the encoder (required).

`tgt_mask` (Optional[Tensor]) – the mask for the tgt sequence (optional).

`memory_mask` (Optional[`Tensor`]) – the mask for the memory sequence (optional).

`tgt_key_padding_mask` (Optional[`Tensor`]) – the mask for the tgt keys per batch (optional).

`memory_key_padding_mask` (Optional[`Tensor`]) – the mask for the memory keys per batch (optional).

`tgt_is_causal` (bool) – If specified, applies a causal mask as tgt mask. Default: False. Warning: `tgt_is_causal` provides a hint that `tgt_mask` is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

`memory_is_causal` (bool) – If specified, applies a causal mask as memory mask. Default: False. Warning: `memory_is_causal` provides a hint that `memory_mask` is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

see the docs in `Transformer`.